Mustafa Al Shaar

Backend foundations, authentication and inventory

Branch: mustafa | Plan begins when the team receives the starter.

Deliver a secure, persisted inventory API. You own schema changes and shared backend security; other members implement their own modules against your stable foundations.

Your code ownership

- apps/api/prisma/**
- apps/api/src/prisma/**
- apps/api/src/modules/auth/**
- apps/api/src/modules/inventory/**
- apps/api/src/common/security.ts

Your agreed interfaces

- POST /auth/login; GET /auth/me
- GET/POST /pharmacist/medicines; PATCH /pharmacist/medicines/:id
- POST /pharmacist/medicines/:id/batches; PATCH /pharmacist/batches/:id
- Reuse Mohammad's pure inventory calculations and Shiam's effective subscription helper.

Start without waiting

Pull the foundation from main into your branch. Run npm ci, copy both apps/api/.env.example and apps/web/.env.example to .env on first setup, then npm run dev. The frontend shows only a plain startup message. No database models/migrations/seed or product features are implemented. Mustafa authors the database setup; follow his instructions after that PR merges.

Read the final specification, proposed API contract, acceptance checklist and your task folders. You build the assigned pages/services yourself. There are no prebuilt guards, calculations, forms, components, adapters or fixtures. Agree interfaces early and create your own small local test data to work independently.

Scope boundary

Do not take over owner/duty/dashboard services or frontend pages. Review colleagues' PRs instead. Do not add optional dispensing, sales, AI or deployment.

Week 1 - build your area

Days are relative, not the instructor deadline. Day 7 is a review/buffer day; adjust to your availability. Deliver small reviewed PRs throughout the week.

Day 1

Run the empty NestJS foundation. Design the Prisma models/relations with the team and agree JWT claims, Actor/guard interfaces and calculation helper signatures. You author the database, migrations, Prisma service and demo seed; none are prebuilt.

Day 2

Implement the initial schema/migration and Prisma service, then build login DTO, bcrypt comparison, JWT issuance and /auth/me. Start a small auth PR; keep schema and guard interfaces stable for teammates.

Day 3

Implement and test JWT, role, ownership and active-subscription write guards plus the business-date utility. Share the imports/interfaces with Shiam and Hamid. Begin seed data; finish it alongside inventory work.

Day 4

Implement medicine create/update/list with trim+case uniqueness, own-pharmacy scoping, query DTOs and database conflict mapping. Use Mohammad's calculation helper for response indicators.

Day 5

Implement batch creation and edit. Allow already expired stock; reject invalid quantity/date/cost. Only quantity, expiry and location are editable. Keep immutable batch fields rejected in PATCH DTO.

Day 6

Wrap medicine/batch mutation and inventoryUpdatedAt in one transaction. Complete name/status filtering, count and multiple indicators; add Swagger response DTOs and meaningful logs.

Day 7

Buffer/review: run API tests, seed and migrate on a clean dedicated database. Merge a usable inventory API by the end of Week 1; record any blockers in the team plan.

Week 2 - integrate and prove

Day 8: API integration. Day 12: core acceptance. Days 13-14: English rehearsal and fixes. No new optional features while CORE is failing.

Day 8

Integrate real inventory/sign-in requests with Alaa. Compare API payloads with shared contracts; fix backend mismatches rather than adding mock fallback.

Day 9

Add regression tests: guessed cross-pharmacy medicine/batch IDs, owner inventory attempt, missing/wrong token, inactive subscription read versus write.

Day 10

Verify duplicate names across pharmacies, duplicate batches, unknown fields, fractional quantities, negative cost and invalid dates. Failed writes must leave inventoryUpdatedAt unchanged.

Day 11

Exercise every inventory indicator filter with Mohammad's fixtures. Verify calendar boundaries and decimal serialization; preserve simultaneous indicators.

Day 12

Review the other backend modules for guard coverage and ownership/privacy errors with their owners. Check persisted data survives restart and migrations apply on another laptop.

Day 13

Run owner->pharmacist->public flow with team. Freeze schema and API shapes; fix only core defects. Review seed coverage and prepare Swagger examples.

Day 14

Buffer/final rehearsal: show login, a forbidden guessed-ID request and database persistence. Explain controller->service->Prisma and why medicine differs from batch in English.

Ready-to-merge checklist

[ ] A01/A02: missing/invalid token and wrong role fail; owner cannot edit inventory; cross-pharmacy read/write fails.

[ ] A05: inactive pharmacists read but cannot change medicines/batches; clear 403 reason.

[ ] A07/A08: normalized duplicate names/batches conflict; bad dates/negative or fractional quantities/immutable PATCH fields rejected.

[ ] A10/A12: filters include every relevant indicator; stock mutation and timestamp are atomic.

[ ] A17: clean migrations, repeatable seed, restart persistence and Swagger match contracts.

Handoffs and dependencies

- Day 1: agree Actor/JWT/guard imports and schema fields. You author these; none is prebuilt.

- Day 2-3: auth/guard PR available for all backend owners.

- Day 2: receive calculation helper from Mohammad; write service tests with fixed sample inputs while waiting.

- Day 6-7: inventory endpoints ready for Alaa. Day 8 begins API integration.

Every pull request

Commit only your intended files; push your branch; open a PR into main. Include the task/rule IDs, a short explanation, and tests or screenshots. Run npm run typecheck, npm test and npm run build. Backend contributors also document their endpoints in Swagger. Alaa reviews/merges; keep .env, node_modules, dist and temporary files untracked.

Your English presentation segment

Show authenticated Swagger inventory access, reject an unauthorized/cross-pharmacy write, then show the persisted PostgreSQL record. Explain one transaction and one uniqueness rule.

Everyone should understand the whole flow: owner onboarding -> pharmacist stock -> dashboard -> public search. Be ready to explain a rule outside your own feature. Final acceptance uses real API requests and persisted PostgreSQL data; locally authored test fixtures are only for development.